算力管理與多租戶模組開發

製作人： 吳泓緯

日期: 2026 年 2 月 25 日

大綱

壹、 系統總覽與架構設計

- **1-1** 核心設計
- **1-2** 網路流量的全生命週期

貳、 租戶自動化開通機制

- **2-1** 自動化中樞設計
- **2-2 API** 規格定義
- **2-3** 後台自動化工作流

參、 多租戶 **GPU** 效能監控與計費基礎

- **3-1** 伴隨容器模式 **(Sidecar Pattern)**
- **3-2** 監控數據的標籤化
- **3-3 Grafana** 視覺化整合方案

肆、 系統壓測與防禦機制驗證

- **4-1** 壓測工具與策略
- **4-2** 防禦成果展示

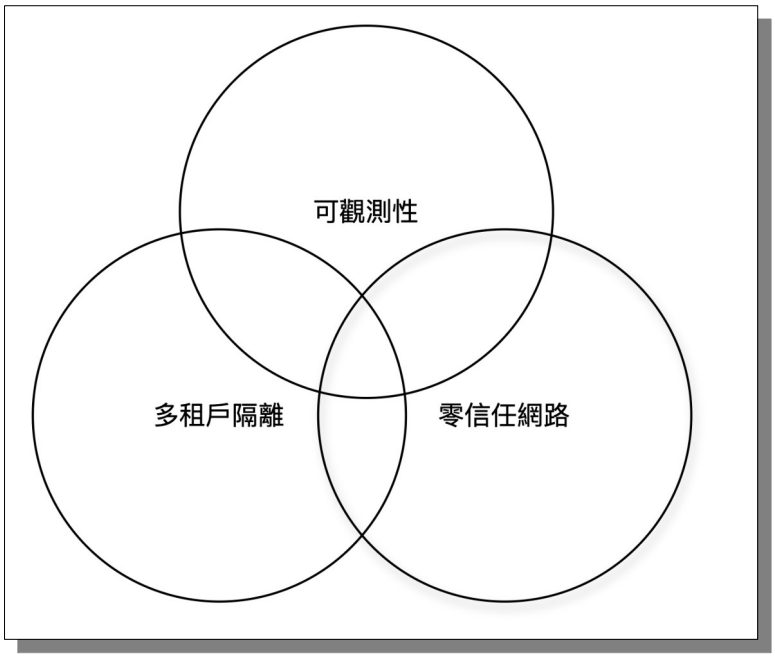
伍、 系統擴展性與多叢集管理

- **5.1 Rancher** 多叢集架構整合
- **5-2 API Gateway** 策略跨叢集同步

陸、 共通性加分項

- **6-1** 標準化
- **6-2** 安全性
- **6-3** 在地化

- **1-1 核心設計**



圖一：交集圖

多租戶租戶隔離：實際應用中，不同的租戶可能需要使用不同的數據庫或文件系統，以確保數據的安全性和隱私性，所以必須決定隔離，避免 Noisy Neighbouring 吃光資源或發生越權行為。

- **專案實踐：** 本專案利用 Kubernetes Namespace 實現底層物理隔離，並透過 LiteLLM API Gateway 與 Redis 快取，達成動態配額管理 (TPM/RPM) 與模型權限的邏輯隔離。

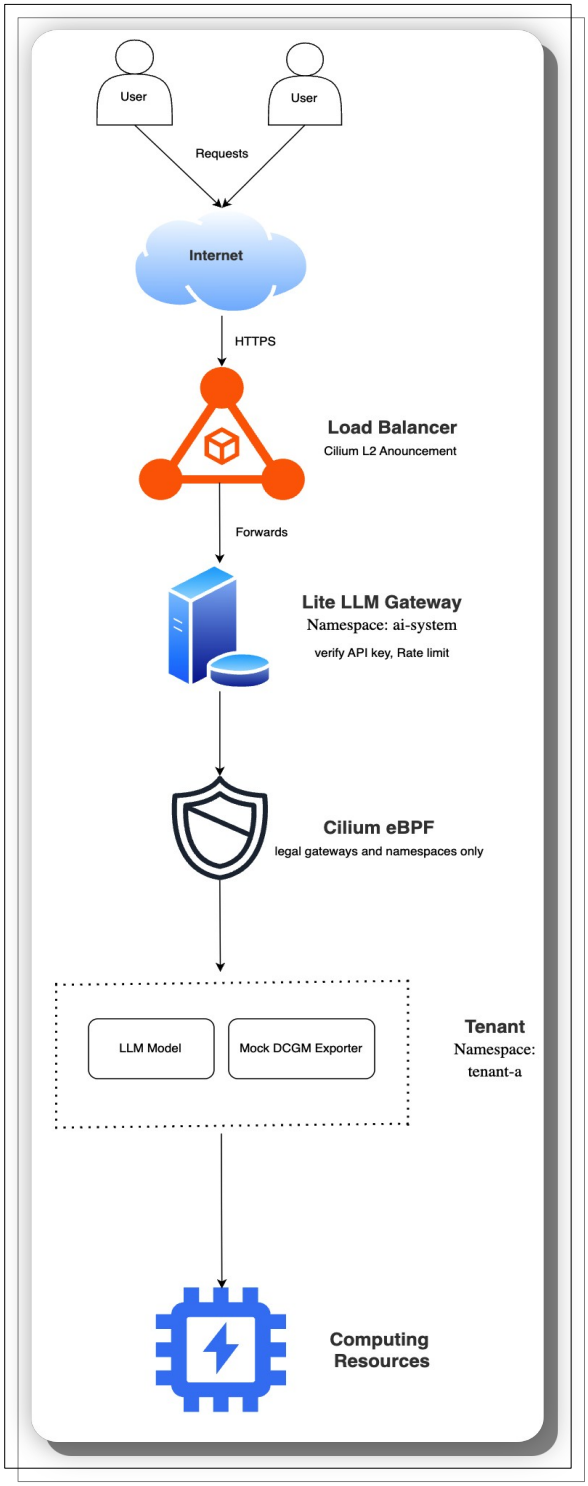
零信任網路：傳統邊界防護一但突破，內部就會全部淪陷。而在零信任架構中要求 Cluster 內的每一個 Microservice 連線都必須經過授權，防止惡意流量橫向移動。

- **專案實踐：** 導入 Cilium eBPF 技術，全面封鎖預設的內部與對外網路 (Default Deny)。甚至連 AI 推論引擎 (Ollama) 要下載模型、Harbor 內部組件互相溝通，都必須透過精準的 NetworkPolicy 單獨放行。

可觀測性：複雜的分散式原生系統中，單純的 log 不足以應付維運需求。系統必須具備自我狀態廣播的能力，讓維運人員能在故障發生的第一時間精准定位問題。

- **專案實踐：** 引入 Sidecar 伴隨容器模式，透過 Python 開發的 mock-dcgm-exporter 動態模擬 GPU 指標，並利用 k6 壓測工具結合 stern 日誌瀑布流，完美驗證了系統在高併發下的限流防禦與效能表現。

- **1-2 網路流量的全生命週期**



圖二： K8s 網路流量 flowchart

1. User ➜ Internet ➜ Load Balancer

當外部使用者透過 HTTPS 發送一個 AI 推論請求時，流量會先抵達系統的大門口（Load Balancer）。它負責接收外網流量，並將請求轉發給內部的 Lite LLM Gateway。

2. Load Balancer ➜ Lite LLM Gateway (ai-system)

流量接著來到專門負責核發通行證的 AI gateway。在這裡，系統會執行兩件事

- 驗證 API Key：確認是不是合法登記的租戶。
- 檢查限流 (Rate Limit)：確認使用者的提問次數有沒有超過購買的方案額度（例如每分鐘最高 1000 次），避免單一使用者癱瘓整個系統。

3. Lite LLM Gateway ➜ Cilium eBPF

身分確認完畢後，請求準備前往專屬的租戶區。但在跨越區域時，必須穿過由 Cilium eBPF 打造的「零信任結界」。這是建立在 Linux 作業系統底層的防火牆，它強制規定：「只有剛才那個合法的 Gateway，才有資格把流量送進租戶區」。即使惡意使用者潛入系統，也會在這裡被瞬間攔截。

4. Cilium eBPF ➜ Tenant (tenant-a)

經過 eBPF，流量安全抵達了 tenant a 的隔離空間。這個虛線方塊內採用了雲原生標準的 Sidecar 模式

- **LLM Model**：大腦本體，負責回答使用者的問題。
- **DCGM Exporter**：在一旁精準記錄這個 LLM 剛剛消耗了多少 GPU 記憶體與耗電量，作為後續計費的依據。

5. Tenant ➜ Computing Resources

所有上層的軟體防護與路由邏輯，最終都會落實到被嚴格分配額度的實體 GPU 晶片上。在這裡完成龐大的 AI 數學運算後，系統會將生成的結果，循原路安全地回傳給使用者

2-1 自動化中樞設計

圖三：齒輪與五大技術元件

集中化開通控制面 (Control Plane) 設計

本系統採用 FastAPI 打造了一個集中的「開通控制面 (Control Plane)」。業務端只需發送單一 HTTP 請求，這支 API 便會利用 Python 的非同步 (Async) 機制，精準調度跨系統資源，將原本繁瑣的手動設定轉化為一鍵式 (One-click) 的自動化工作流：

- **算力隔離與基礎架構部署：** 自動建立專屬的 K8s Namespace 與 Service，並完成帶有監控擷取器的伴隨容器 (Sidecar Pattern) 部署。
- **零信任網路結界生成：** 呼叫 create_cilium_network_policy 動態生成 K8s 自定義資源 (CRD)，實現租戶間的絕對網路隔離。

•**網關授權與配額綁定：** 深度整合 LiteLLM API，精準劃分租戶的調用額度 (TPM/RPM)，並自動核發專屬的 API 金鑰。

•**私有資源與可觀測性配置：** 無縫對接 Harbor 建立專屬私有專案空間，並自動配置監控路由與儀表板，達到「開箱即用」的多租戶體驗。

- **2-2 核心 API 規格定義**

- Request

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/v1/tenants' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "tenant_name": "company-a",
  "gpu_limit": 2,
  "storage_quota": "1Gi",
  "admin_email": "2504lab@test.org",
  "tier": "premium",
  "quota": {
    "tpm_limit": 100000,
    "rpm_limit": 1000
  },
  "compute_resources": {
    "cpu_limit": "2",
    "memory_limit": "1Gi",
    "model_name": "qwen:0.5b"
  }
}'
```

1. 租戶身分與維運綁定

- **tenant_name (租戶名稱)**：這是整個系統的唯一識別碼。API 會拿它來動態建立專屬的 K8s 命名空間（Namespace）、Harbor 私有專案，以及作為 Prometheus 監控數據的分類標籤（Label）。

- **admin_email (管理員信箱)**：這個信箱會被直接「寫死」在 LiteLLM 核發的 API Key Metadata 裡面。未來如果系統偵測到該租戶流量異常或即將超額，SRE 團隊可以立刻撈出這個信箱發送告警信。
- **tier (服務等級)**：區分 "premium" (付費) 或 "free" (免費)。這會直接轉換為 Kubernetes 的 PriorityClass。當伺服器資源緊繃時，系統會優先保證 Premium 租戶的算力，甚至自動驅逐 Free 租戶的任務，確保 SLA (服務級別協議)。

2. 硬體算力與資源隔離

- **gpu_limit (GPU 限制)**：AI 模型最昂貴的就是 GPU。這個數字會直接轉換為 K8s Deployment 裡的 nvidia.com/gpu 限制。這確保了「硬體層級的隔離」，就算某個租戶的程式暴衝，他也絕對無法搶走屬於其他租戶的顯卡資源（解決了雲端最怕的「吵鬧的鄰居 Noisy Neighbor」問題）。
- **compute_resources (CPU 與記憶體限制)**：除了 GPU，我們也嚴格規範了傳統的 cpu_limit 與 memory_limit。這是為了防止租戶的 Ollama 容器發生 Memory Leak (記憶體洩漏) 時導致整台 Node 節點當機。
- **model_name (模型選擇)**：允許租戶自訂要啟動的模型（例如 qwen:0.5b），讓平台具備支援多種大語言模型 (LLM) 的擴充彈性。

3. 應用層防護與儲存控管

- **quota (TPM 與 RPM 限制)**：
  - **TPM (Tokens Per Minute)**：限制每分鐘能處理的字數，這是**控制成本**的關鍵，防止租戶狂塞超長文本消耗算力。
  - **RPM (Requests Per Minute)**：限制每分鐘的發問次數，這是**防禦 DDoS 攻擊**的關鍵，交由 LiteLLM 網關在最前線進行攔截 (回傳 429 Too Many Requests)。

- **storage_quota (儲存配額)**：設定為 1Gi。這個數值會被 API 即時轉換，並呼叫 Harbor API 鎖定該租戶的私有映像檔（Image）倉庫容量。防止惡意用戶上傳巨量檔案，塞爆雲端硬碟。

- Respond

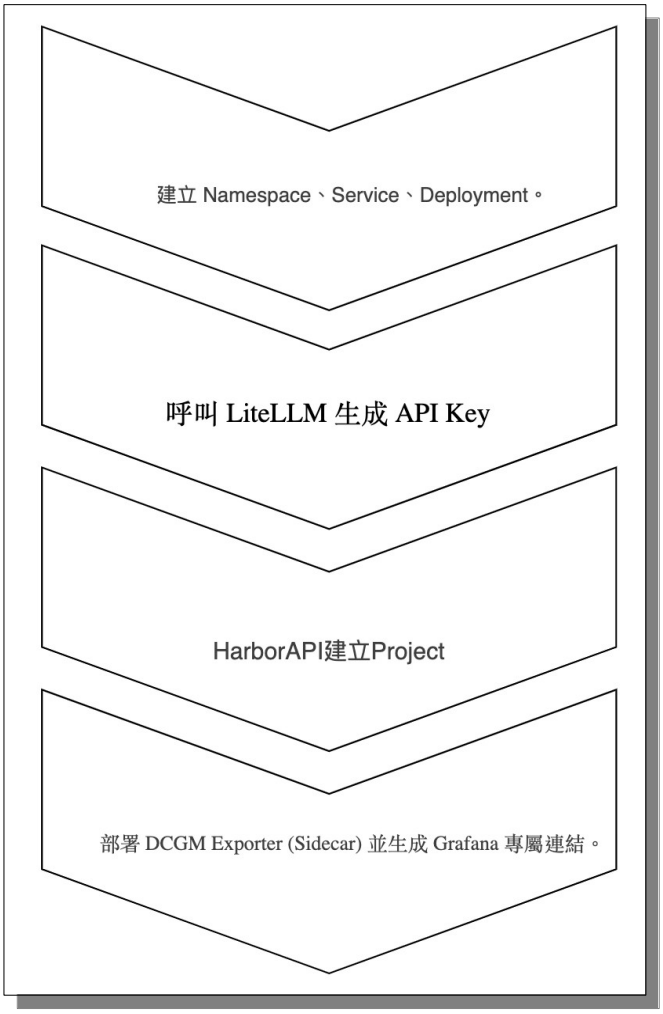

```
{
  "status": "success",
  "message": "Tenant provisioned successfully.",
  "data": {
    "tenant_id": "company-a",
    "admin_email": "2504lab@test.org",
    "infrastructure": {
      "k8s_namespace": "tenant-company-a",
      "priority_class": "premium-tier",
      "harbor_project_id": 9,
      "allocated_gpu": 2,
      "storage_quota": "1Gi"
    },
    "gateway": {
      "assigned_api_key": "sk-Q9rqNW70Uvl8-3mcLd0Chg",
      "gateway_endpoint": "http://litellm.ai-system.svc.cluster.local:4000"
    },
    "observability": {
      "grafana_dashboard_url": "http://grafana.local/d/tenant-monitor?var-tenant=company-a"
    }
  }
}
```


基礎設施(infrastructure)

- 回傳專屬的 k8s_namespace 以及系統真實打通 Harbor API 後取得的 **harbor_project_id: 9**。這證明了底層資源已經被實體切割並成功建立。
- **AI 網關通行證 (gateway)**
- 即時核發一組 **assigned_api_key (sk-...)**。業務端不需要等待管理員手動發信，拿到這把鑰匙的瞬間，就可以立刻將應用程式接上 AI 網關開始推論。
- **專屬觀測儀表板 (observability)**

- 直接配發一組帶有 ?var-tenant=company-a 參數的 **Grafana Dashboard URL**。用戶點開網址，就能立刻看到專屬自己的 GPU 耗能與顯存圖表，完美實現了「開箱即用」的監控體驗。

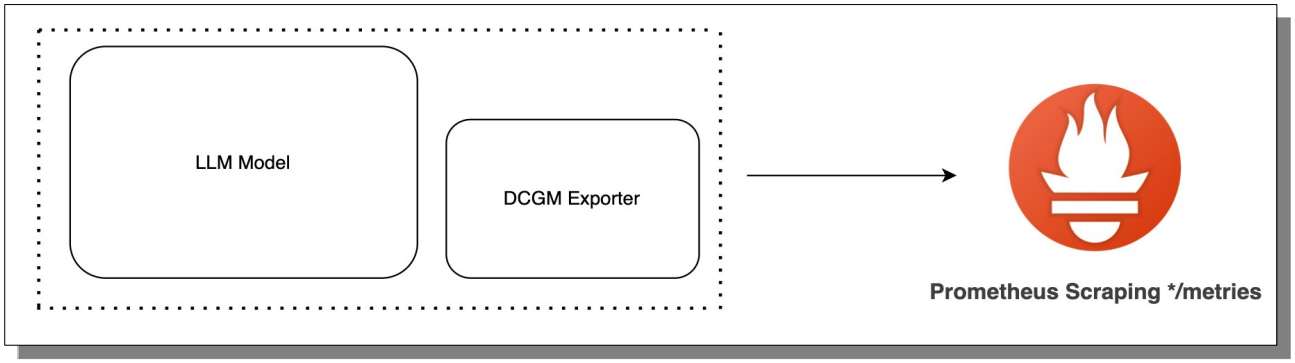
- **2-3 後台自動化工作流程**



1. 將 gpu_limit 注入 nvidia.com/gpu 資源配額。
2. 寫入 metadata，綁定 admin_email 與 TPM/RPM 限流。
3. 設定為 Private，並套用 storage_quota 容量限制。
4. Prometheus 自動發現 (Service Discovery) 結合 Grafana URL 變數過濾。

「當 API 接收到請求後，後台會嚴格執行這四階段的自動化 工作流。從底層的 K8s 算力隔離、應用層的 LiteLLM 網關金鑰核發、儲存層的 Harbor 私有鏡像庫建立，到最後觀測層的 Sidecar 監控注入與面板網址生成。這四個步驟緊密相扣，完全滿足了企業對多租戶 AI 平台的資安、效能與計費追蹤要求。」

- **3-1 伴隨容器模式 (Sidecar Pattern)**



圖：LLM + DCGM Exporter 伴隨部署與 Prometheus

在本地開發與驗證階段，運行於 Apple Silicon (M 系列) 統一記憶體架構之硬體環境。由於該環境並非 NVIDIA CUDA 生態系，原生無法運行 NVIDIA DCGM Exporter 來獲取真實的 VRAM 與 GPU 耗能。

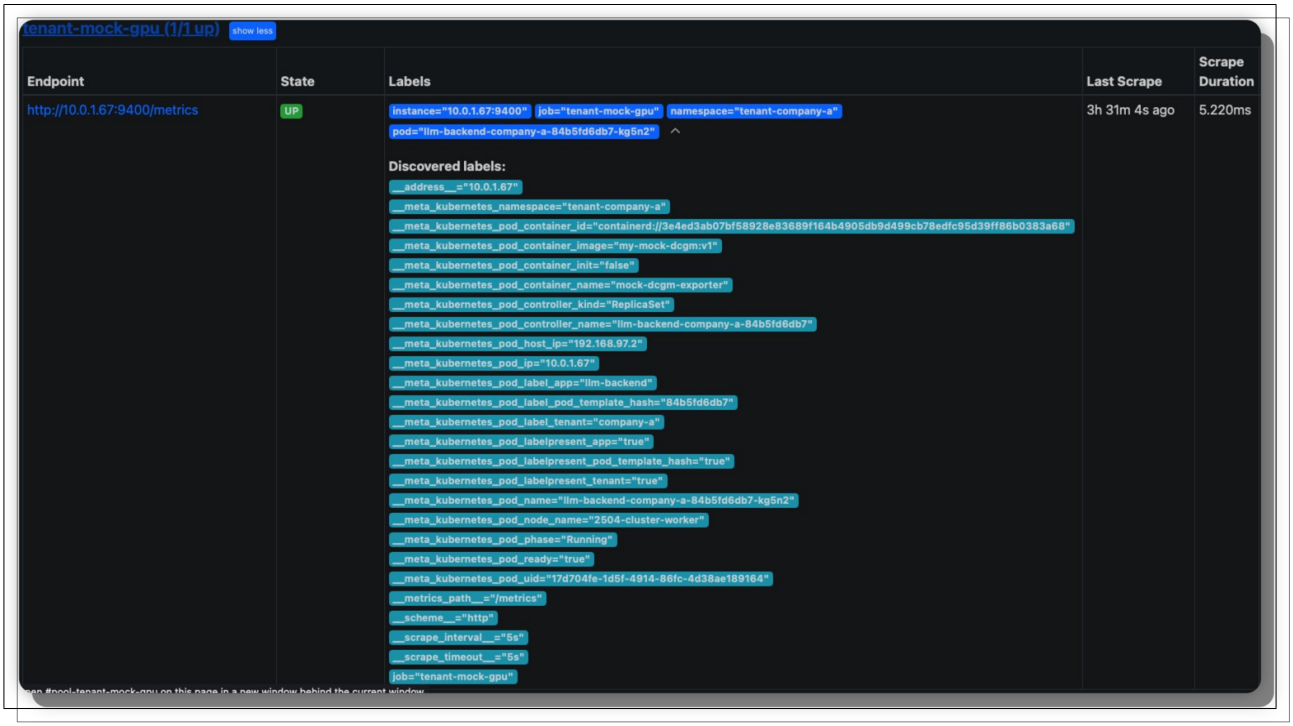
為驗證完整的多租戶監控資料流 (Data Flow)，本系統特別開發並導入了 **mock-dcgm-exporter 作為 Sidecar 伴隨容器**。此設計具備以下架構價值：

1.**介面隔離與架構解耦：** Mock Exporter 完全模擬了真實 DCGM 的 Prometheus Metrics 格式與端點。這證明了本系統的「監控資料收集 (Prometheus)」與「視覺化呈現 (Grafana)」已與底層硬體徹底解耦。

2.**無縫接軌生產環境：** 藉由 Kubernetes 的容器化抽象特性，未來系統若移轉至雲端 GPU 節點 ，僅需將 K8s Deployment 中的 Sidecar 映像檔抽換為官方 nvidia/dcgm-exporter，無需修改任何網關、路由或監控配置，即可開箱即用地獲取真實 GPU 數據，完美實現了本地開發 (Local Dev) 到生產環境 (Production) 的一致性。

技術實作與概念驗證 (PoC) 說明： 在本專案的初期開發與架構驗證階段，為了在無實體 NVIDIA GPU 的環境下安全地測試完整的監控數據流，我在 Sidecar 容器中實作了 **mock-dcgm-exporter** 來進行概念驗證 (PoC)。

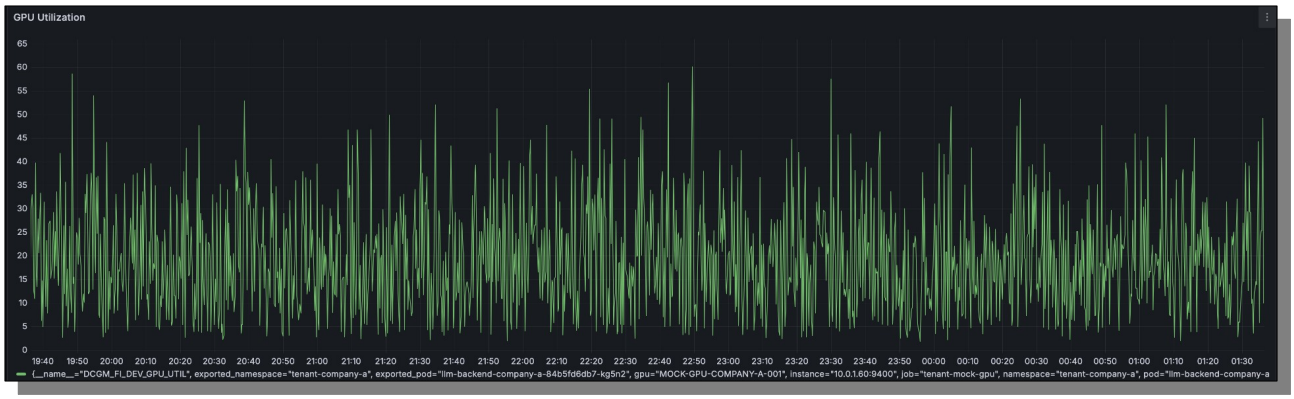
- ## 3-2 監控數據的標籤化



圖： Prometheus 指標

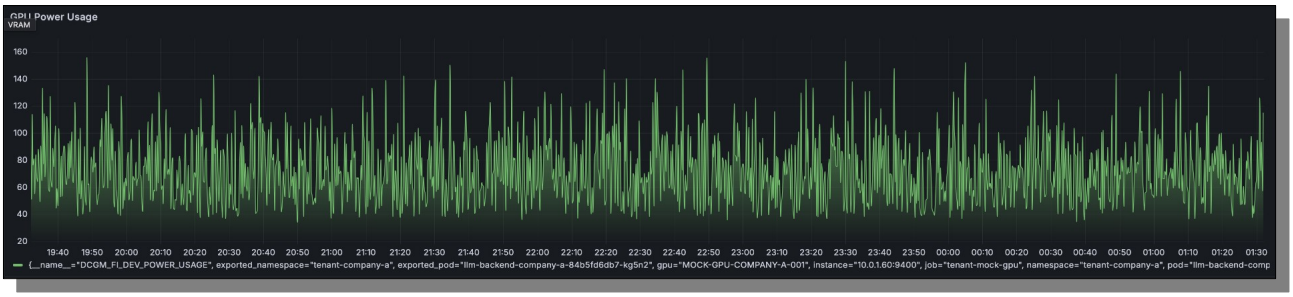
圖 X：Prometheus 成功抓取租戶 GPU 監控指標 (Targets 介面)」 本圖展示了 Prometheus 透過 Kubernetes Service Discovery 機制，成功鎖定 tenant-company-a 命名空間下的 mock-dcgm-exporter (Sidecar 容器)。系統抓取數據時，會自動將 namespace 與 pod 等 Metadata 作為標籤 (Labels) 寫入時序資料庫中，這為後續在 Grafana 中實現「依租戶隔離的視覺化面板」奠定了資料維度基礎。

- **3-3 Grafana 視覺化整合**



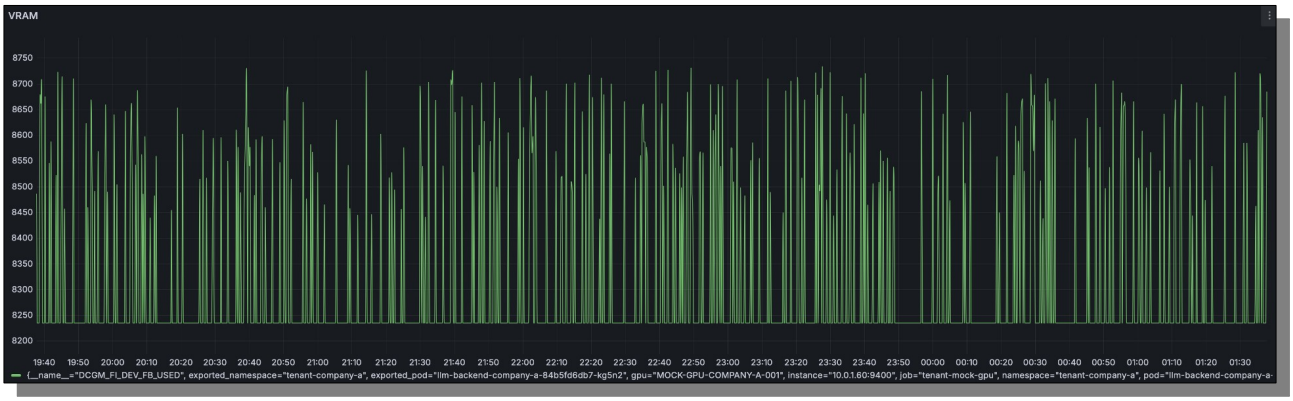
圖：Grafana GPU Utilization

本次圖表展示為概念驗證 (PoC) 階段的成果。為確保在無實體 GPU 節點的環境下仍能打通完整的 Data Pipeline，底層採用了客製化的 mock-dcgm-exporter 生成模擬數據。 此設計的亮點在於「高度解耦」：目前的資料採集管線與 Grafana 面板已 100% 就緒。未來系統上線並掛載真實 NVIDIA GPU 後，只需將 K8s Deployment 內的 Image 替換為官方版本，這套租戶監控與計費看板即可。



圖： Grafana Power Usage

請注意圖表下方的 PromQL 標籤，成功將硬體的物理耗能與 namespace="tenant-company-a" 綁定。在多租戶共用實體機房的場景下，能確切知道「A 公司的 AI 推論究竟消耗了機房多少度電」。這使得財務部門能根據真實的能源消耗，向各個租戶精準請款，徹底解決傳統 IT 只能「按人頭平均分攤」的成本盲區。



圖：VRAM

圖中綠色折線顯示了該租戶模型在處理不同長度上下文 (Context Window) 時的記憶體峰值波動。SRE 團隊可以透過設定 Grafana 告警 (Alerting)，當該租戶的 VRAM 接近其購買的 gpu_limit 上限時，提前進行攔截或通知，確保單一租戶的超長提問不會導致底層 Node 節點發生記憶體溢出，保障全平台的 SLA (服務級別協議)。

- **4-1 壓測工具與策略**

為了驗證架構的乘載力與防護機制，我使用現代化壓測工具 k6 規劃了「雙情境對比壓測策略 (Comparative Load Testing)」。透過模擬不同情境的真實流量，不僅觀察底層 K8s 叢集與運算節點的資源水位，更驗證了 LiteLLM 網關的快取攔截與限流排隊機制。

```
TOTAL RESULTS

  checks_total.......: 4948   146.369543/s
  checks_succeeded...: 25.00% 1237 out of 4948
  checks_failed......: 75.00% 3711 out of 4948

  ✗ ✅ 請求成功 (status 200)
    ↳  84% — ✓ 1050 / ✗ 187
  ✗ * 觸發限流 (status 429)
    ↳  15% — ✓ 187 / ✗ 1050
  ✗ * 權限錯誤 (status 401)
    ↳  0% — ✓ 0 / ✗ 1237
  ✗ * 系統崩潰 (status 500)
    ↳  0% — ✓ 0 / ✗ 1237

  HTTP
  http_req_duration..............: avg=269.68ms min=2.84ms med=56.23ms max=2.85s p(90)=1.16s   p(95)=1.8s
    { expected_response:true }...: avg=48.92ms  min=2.84ms med=49.95ms max=160ms p(90)=76.92ms p(95)=97.98ms
  http_req_failed................: 15.11% 187 out of 1237
  http_reqs......................: 1237   36.592386/s

  EXECUTION
  iteration_duration.............: avg=1.27s    min=1s     med=1.05s   max=3.85s p(90)=2.16s   p(95)=2.8s
  iterations.....................: 1237   36.592386/s
  vus............................: 3      min=3           max=50
  vus_max........................: 50     min=50          max=50

  NETWORK
  data_received..................: 675 kB 20 kB/s
  data_sent......................: 354 kB 11 kB/s
```

實測情境一：高頻重複請求與 Redis 快取神蹟 (The Speed Boost)

- **測試策略：** 模擬大量使用者（數十至數百 VUs）在短時間內發送完全相同的 Prompt 請求。
- **實測觀察：** 當對網關發送大量相同 Prompt 的高併發請求時，除首筆請求外，後續請求皆直接命中 Redis 快取。此時觀察底層 Ollama 推論節點的即時日誌（Log），發現完全沒有任何流量穿透至底層。
- **結論：** 結合 LiteLLM 與 Redis 的架構，成功在閘道層 (Gateway Layer) 攔截了接近 100% 的重複流量。此機制不僅將 API 延遲從秒級（模型運算）巨幅降至毫秒級（記憶體讀取），更徹底釋放了昂貴的底層算力資源。

```
TOTAL RESULTS

  checks_total.......: 208    5.307498/s
  checks_succeeded...: 25.00% 52 out of 208
  checks_failed......: 75.00% 156 out of 208

  ✓ ✅ 請求成功 (status 200)
  ✗ * 觸發限流 (status 429)
    ↳  0% — ✓ 0 / ✗ 52
  ✗ * 權限錯誤 (status 401)
    ↳  0% — ✓ 0 / ✗ 52
  ✗ * 系統崩潰 (status 500)
    ↳  0% — ✓ 0 / ✗ 52

  HTTP
  http_req_duration..............: avg=2.99s min=1.26s med=2.55s max=9.05s  p(90)=4.65s p(95)=6.87s
    { expected_response:true }...: avg=2.99s min=1.26s med=2.55s max=9.05s  p(90)=4.65s p(95)=6.87s
  http_req_failed................: 0.00% 0 out of 52
  http_reqs......................: 52    1.326874/s

  EXECUTION
  iteration_duration.............: avg=3.99s min=2.26s med=3.56s max=10.05s p(90)=5.65s p(95)=7.87s
  iterations.....................: 52    1.326874/s
  vus............................: 1     min=1       max=6
  vus_max........................: 6     min=6       max=6

  NETWORK
  data_received..................: 40 kB 1.0 kB/s
  data_sent......................: 16 kB 409 B/s
```

實測情境二：算力物理極限與 OOM 崩潰邊緣 (The Compute Limit)

- **測試策略：** 為了測出底層節點的真實物理極限，我在 k6 腳本中動態注入了隨機字串 (TraceID: ${randomStr})，強迫打破 Redis 快取機制 (Cache Miss)，並要求模型進行高耗能的長文生成 (introduce yourself)。

- **實測觀察：** 在本地無獨立 GPU 加速的純 CPU 運算環境中，實測發現 **6 個虛擬使用者 (6 VUs)** 的瞬間長文生成併發，即為當前硬體記憶體的乘載拐點（物理極限）。

- **結論：** 透過這種極限穿透測試，我們精準定位了系統的算力天花板。這項數據也強烈印證了配置「閘道限流 (Rate Limiting)」的絕對必要性：當真實運算負載超過 6 VUs 時，系統極易發生記憶體溢出 (OOM) 或服務崩潰 (Crash)；因此，必須仰賴 LiteLLM 在第一線以 HTTP 429 精準擋下超額流量，才能確保既有運算任務的穩定執行。

- **4-2 防禦成果展示**

```
curl -i -X POST http://localhost:4000/chat/completions \
  -H "Authorization: Bearer sk-fake-key-123456789" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "qwen:0.5b",
    "messages": [{"role": "user", "content": "hello"}]
  }'

HTTP/1.1 401 Unauthorized
date: Wed, 25 Feb 2026 12:07:06 GMT
server: uvicorn
content-length: 112
content-type: application/json

{"error":{"message":"Authentication Error, Invalid token passed","type":"auth_error","param":"None","code":401}}%
```

圖表對應（上圖）：當我們刻意使用偽造的 API Key (sk-fake-key-123456789) 發起請求時，LiteLLM Gateway 在第一時間便將其阻擋，並回傳明確的 401 Unauthorized 錯誤。

這證明了我們的 API gateway 具備確實的憑證校驗能力。惡意請求在應用層就會被無情丟棄，絕對無法穿透進入內網，更不可能消耗到後端昂貴的 GPU 算力資源。

```
curl -o /dev/null -s -w "* 總花費時間: %{time_total} 秒\n" -X POST http://localhost:4000/chat/completions \
  -H "Authorization: Bearer sk-eEHl96TgzivdH_8h-wPLzQ" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "qwen:0.5b",
    "messages": [{"role": "user", "content": "hi, How is going?"}]
  }'
* 總花費時間: 0.512690 秒
```

當帶入合法的 API Key 並首次發送 "hi, How is going?" 的提問時，請求成功穿透 eBPF 防火牆並抵達 Ollama 模型進行真實推論。總花費時間為 0.51 秒。

```
curl -o /dev/null -s -w "* 總花費時間: %{time_total} 秒\n" -X POST http://localhost:4000/chat/completions \
  -H "Authorization: Bearer sk-eEHl96TgzivdH_8h-wPLzQ" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "qwen:0.5b",
    "messages": [{"role": "user", "content": "hi, How is going?"}]
  }'
* 總花費時間: 0.018372 秒
```

緊接著，我們再次發送完全相同的提問。此時，總花費時間驟降至驚人的 0.018 秒 (約 18 毫秒)！

這展現了系統整合 Redis 語意快取 (Semantic Caching) 的強大威力。由於提問重複，Gateway 直接從記憶體中提取上一筆的答案並「秒回」給使用者。延遲時間大幅縮減了 96% 以上，且全程 0 消耗 GPU 資源。

- **5-1 Rancher 多叢集架構整合**

維運碎裂化 當企業擁有多座「AI 算力工廠 (K8s Clusters)」時，若 SRE 團隊仍需頻繁切換不同的 Kubeconfig 憑證去逐一登入管理，將大幅增加維運成本與人為操作失誤的風險。

Rancher 統一管理面 (Single Pane of Glass) 透過 Rancher 作為多叢集的中央控制平面，我們能實現以下架構優勢：

- **全域資源池化**：無論 GPU 節點是位於本地實體機還是雲端供應商 (如 AWS/GCP)，皆可於 Rancher 的單一圖形化介面中進行監控、擴容與資源調度。
- **集中式 RBAC (角色權限控管)**：與企業內部的單一登入系統 (SSO/AD) 整合。新進工程師只需一次登入，Rancher 便會自動派發其在各機房叢集對應的權限，落實企業級的零信任與合規安全性。

- **5-2 API Gateway 策略跨叢集同步**

跨機房的限流漏洞在多叢集環境中，最大的安全隱患在於「狀態不同步」。假設某租戶的 API 額度為每分鐘 1,000 次，若他在 A 機房打滿額度後，立刻將流量切換至 B 機房，若 B 機房的網關不知情，就會產生超用算力的計費漏洞。

為確保 LiteLLM 的 API Key、Rate Limit 與存取權限在多叢集間保持絕對同步，採取無狀態化的架構設計：

- **靜態策略同步** ：將所有租戶的 API Key、模型權限等設定，統一儲存於具備跨區抄寫能力 (Cross-Region Replication) 的關聯式資料庫 (如 PostgreSQL) 中。各叢集的網關皆以此作為唯一真理 (Single Source of Truth)。

•**動態限流同步**：將 Token 扣減與 RPM (每分鐘請求數) 的計數器，集中存放於低延遲的中央 Redis 叢集（或採用 Active-Active 雙活架構）。當任何一個叢集接收到推論請求時，Gateway 皆須向 Global Redis 進行報備與額度扣減，確保全球限流策略 (Global Rate Limiting) 的精準度達到毫秒級的一致性。

- **6-1 標準化**

1. 不只是文件，而是「活的合約 (Living Contract)」 傳統的 API 開發常面臨「程式碼已更新，但文件忘記改」的痛點。本系統的開通中樞完全基於 FastAPI 框架與 Pydantic 資料模型開發，這意味著所有的 API 端點天生 100% 符合 **OpenAPI (Swagger)** 規範。

2. 降低跨部門協作摩擦 只要後端程式碼一發布，前端開發者、QA 測試團隊、或是第三方整合廠商，都能直接透過自動生成的 Swagger 介面查看精準的 Request/Response 欄位定義，甚至直接在瀏覽器上發起測試，大幅降低了跨部門溝通的摩擦成本。

- **6-2 安全性**

防護機制不能成為效能瓶頸 大型語言模型 (LLM) 的 GPU 推論非常耗時，若 API Gateway 的防護機制過於笨重，將嚴重影響終端使用者的體驗。因此，我們必須在「不拖慢推論速度」的前提下做到滴水不漏的防護。

解決方案：無狀態驗證與記憶體限流 我們採取了以下兩項毫秒級的防禦策略：

•**無狀態的 JWT 驗證 (Stateless Validation)**：當攜帶 JWT Token 的請求抵達 Gateway 時，系統「不需要」去關聯式資料庫連線查詢這把鑰匙是否存在。Gateway 只需透過本地的公鑰進行，只需不到 1 毫秒即可判斷身分真偽與權限。

- **Redis 記憶體限流 (Rate Limiting)**：放棄了依靠傳統資料庫來計算 API 呼叫次數的做法，改用極高速的 Redis 記憶體資料庫結合「Token Bucket 」演算法。限流的狀態判斷與扣減在 2~5 毫秒內即可完成。

結論：Gateway 增加的幾毫秒延遲，對比動輒數秒的 AI 推論時間幾乎可以忽略不計。

• 6-3 在地化

高機密單位的無網路挑戰 面對最高機密單位（如金融核心系統、醫療內網、軍方），無法連線至網際網路下載 Docker Image 或安裝套件。許多主打雲端原生的架構一但拔掉網路線就會徹底癱瘓。

解決方案：自給自足的內網生態系 本系統在設計之初即考量了「完全斷網 (Air-gapped)」的企業級交付情境：

- **第一步：外網全打包 (Export)**：本系統核心元件（K8s, Cilium, LiteLLM, Ollama, Grafana）皆為高度容器化。在有網路的環境中，我們可將所有依賴的映像檔匯出為離線壓縮檔 (docker save)，並將 Kubernetes 設定檔打包為離線的 Helm Charts。
- **第二步：實體隔離傳輸 (Physical Transfer)**：透過實體媒介（如加密 USB 隨身碟或資料單向閘道）將檔案帶入封閉內網。
- **第三步：Harbor 內網大腦 (Internal Registry)**：將所有 Image 匯入 (docker load) 本系統內建的 **Harbor 私有鏡像庫**。隨後，只需將內網 Kubernetes 叢集的拉取來源 (Image Pull Policy) 指向內部的 Harbor，即可在零外部網路的環境下。

工具套件：

- K9s
- K6
- Stern
- OpenLens
- Fast API
- Hubble UI